La gestion d'erreur

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte 3

II. Les erreurs et exceptions 3

A. Les erreurs et exceptions 3

B. Exercice : Quiz 5

III. Gérer les erreurs en JavaScript 5

A. Gérer les erreurs en JavaScript 5

B. Exercice : Quiz 8

IV. Essentiel 8

V. Auto-évaluation 9

A. Exercice 9

B. Test 11

Solutions des exercices 12

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 heure

Environnement de travail : VS Code

Prérequis : aucun

Contexte

La gestion des erreurs est une compétence cruciale pour tout développeur. Les erreurs peuvent survenir à tout moment et leur impact peut être significatif. Une erreur non détectée entraîne rapidement des problèmes importants, tels que des plantages d'applications, des comportements inattendus ou même des failles de sécurité.

Dans ce cours, nous allons explorer les différentes techniques et outils de gestion des erreurs en JavaScript. Nous allons commencer par comprendre les différents types d'erreurs qui peuvent se produire en JavaScript, comment les identifier et les résoudre efficacement. Nous verrons également comment utiliser les blocs try-catch pour gérer les erreurs de manière proactive.

II. Les erreurs et exceptions

A. Les erreurs et exceptions

Fondamental

En JavaScript, les **erreurs** sont un type d'objet créé à l'aide de la syntaxe `new Error()`. Elles sont traitées comme des objets et peuvent être retournées ou placées dans des variables. L'exécuteur JavaScript lui-même renvoie des erreurs.

Une **exception** est une erreur levée. Elle peut être levée par le contexte d'exécution ou par le code avec le mot-clé `throw`, et ne peut plus être manipulée. Elle met ainsi fin aux contextes d'exécution les uns après les autres en remontant, jusqu'à afficher une erreur dans la console. Elle peut être interceptée avec try, et l'erreur qu'elle remonte peut être manipulée via catch (exception). C'est ce que nous verrons dans ce cours.

En d'autres termes, une erreur est un danger qui pourrait arriver. La classe Error nous donnera un outil pour gérer l'erreur si elle se produit. Quand l'erreur se produit, on l'appelle « *exception* ».

Les erreurs de syntaxe

Les erreurs de syntaxe se produisent lorsqu'une instruction est incorrecte syntaxiquement. Elles sont souvent signalées par des messages d'erreur dans la console du navigateur. **Pour corriger une erreur de syntaxe, il faut modifier votre code.**

Exemple

Exemple de code avec une erreur de syntaxe :

```
1 const x = 10;
2 if (x === 10 {
3   console.log("x est égal à 10");
4 else {
5   console.log("x n'est pas égal à 10");
6 }
```

L'erreur dans ce code est une parenthèse manquante après le `if`. Le message d'erreur dans la console sera :

```
1 Uncaught SyntaxError: Unexpected token 'else'
```

Les erreurs d'exécution

Les erreurs d'exécution se produisent lorsqu'une instruction ne peut pas être exécutée correctement. Elles peuvent être causées par des valeurs incorrectes ou inattendues. **Pour corriger une erreur d'exécution, il faut modifier votre code.**

Exemple

Exemple de code avec une erreur d'exécution :

```
let x = 10;
let y = x.toUpperCase();
console.log(y);
```

Le problème ici est dû au fait que la méthode `toUpperCase` ne peut pas être appelée sur un nombre. Le message d'erreur dans la console sera :

```
Uncaught TypeError: x.toUpperCase is not a function
```

Les erreurs tierces

Les erreurs de bibliothèque se produisent lorsque nous utilisons une bibliothèque tierce qui ne fonctionne pas correctement ou qui renvoie des erreurs. Ces erreurs peuvent être difficiles à diagnostiquer, car elles sont souvent liées à la bibliothèque et non à votre code.

Exemple

Exemple de code avec une erreur de bibliothèque :

```
// Utilisation de la bibliothèque moment.js
let date = moment("2022-12-31T23:59:59");

// Utilisation de la méthode format qui renvoie une erreur
let dateStr = date.format("DD-MM-YYYY");
console.log(dateStr);
```

Dans cet exemple, nous utilisons la bibliothèque « *moment.js* » pour manipuler des dates. Nous tentons d'utiliser la méthode `format`, mais elle génère une erreur. Dans ce cas, il est important de consulter la documentation de la bibliothèque et de vérifier que nous utilisons correctement ses méthodes.

Les erreurs de réseau

Les erreurs de réseau se produisent lorsqu'une requête réseau ne peut pas être exécutée correctement. Elles peuvent être causées par une connexion lente, une erreur de serveur ou un serveur inaccessible. Ce sont des erreurs très fréquentes, il est donc fondamental de savoir les gérer.

Exemple

Exemple de code avec une erreur de réseau :

```
// Utilisation de la méthode fetch pour récupérer des données
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error("Une erreur s'est produite : ", error));
```

Dans cet exemple, nous utilisons la méthode `fetch` pour récupérer des données à partir d'une API en ligne. Si la requête échoue pour une raison quelconque, le bloc `catch` s'exécutera et affichera un message d'erreur dans la console.

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice : Quiz

[solution n°1 p.13]

Question 1

Qu'est-ce qu'une erreur en JavaScript ?

- ○ Une fonctionnalité de JavaScript qui permet de contrôler le flux d'exécution du code
- ○ Un message d'avertissement qui s'affiche lorsqu'une opération est effectuée avec des données non valides
- ○ Un arrêt de l'exécution du code en raison d'un comportement imprévu ou d'une condition invalide

Question 2

Les erreurs de syntaxe en JavaScript se produisent lorsque nous avons fait une erreur dans l'écriture de notre code (faute de frappe, mot-clé qui n'existe pas dans le langage, etc.).

- ○ Vrai
- ○ Faux

Question 3

Qu'est-ce qu'une erreur tierce en JavaScript ?

- ○ Une erreur qui se produit dans notre propre code
- ○ Une erreur qui se produit dans une bibliothèque tierce que nous utilisons dans notre code
- ○ Une erreur qui se produit dans le navigateur du client

Question 4

On peut résoudre une erreur tierce dans notre propre code.

- ○ Vrai
- ○ Faux

Question 5

Quelle est la différence entre une erreur de syntaxe et une erreur d'exécution en JavaScript ?

- ○ Une erreur de syntaxe se produit lorsqu'une instruction est mal écrite, tandis qu'une erreur d'exécution se produit lorsqu'une instruction est exécutée avec des données non valides
- ○ Une erreur de syntaxe se produit lorsqu'une instruction est exécutée avec des données non valides, tandis qu'une erreur d'exécution se produit lorsqu'une instruction est mal écrite
- ○ Il n'y a pas de différence entre les 2 types d'erreurs

III. Gérer les erreurs en JavaScript

A. Gérer les erreurs en JavaScript

Try...catch

La structure `try...catch` permet d'exécuter une instruction tout en gérant les erreurs potentielles. Le bloc `try` contient le code à exécuter, tandis que le bloc `catch` contient le code à exécuter en cas d'erreur.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Exemple de code avec `try...catch` :

```
try {
  let x = 10;
  let y = x.toUpperCase();
  console.log(y);
} catch (error) {
  console.log("Une erreur s'est produite : " + error.message);
}
```

On voit ici que le bloc `try` contient 2 instructions. La première affecte la valeur 10 à la variable x. La deuxième essaie de convertir la valeur de x en majuscules, ce qui provoque une erreur. Le bloc `catch` capture l'erreur et affiche un message personnalisé dans la console.

throw

La structure `throw` permet de générer une erreur personnalisée. Elle est souvent utilisée pour signaler des erreurs spécifiques à l'application.

Exemple

Exemple de code avec `throw` :

```
function diviser(a, b) {
  if (b === 0) {
    throw new Error("Division par zéro impossible");
  }
  return a / b;
}

try {
  let resultat = diviser(10, 0);
  console.log(resultat);
} catch (error) {
  console.log("Une erreur s'est produite : " + error.message);
}
```

Dans cet exemple, nous avons une fonction `diviser` qui effectue une division entre 2 nombres. Si la valeur de b est égale à 0, nous lançons une erreur personnalisée avec `throw`. Lorsque la fonction diviser est appelée dans le bloc try, elle lève une erreur, car la valeur de b est 0. Le bloc `catch` capture l'erreur et affiche un message personnalisé dans la console.

Les erreurs personnalisées

Il est important de noter que `throw` peut générer n'importe quelle erreur, pas seulement une instance de Error. Nous pouvons également créer des classes personnalisées qui étendent la classe Error pour définir des erreurs personnalisées avec des propriétés et des méthodes spécifiques à notre application.

Exemple

Exemple de code avec une classe d'erreur personnalisée :

```
class AgeInvalideError extends Error {
  constructor(message) {
    super(message);
    this.name = "AgeInvalideError";
    this.date = new Date();
```


Tous droits réservés à STUDI - Reproduction interdite

```
  }
}

function verifierAge(age) {
  if (age < 18) {
    throw new AgeInvalideError("L'âge doit être supérieur ou égal à 18 ans");
  }
  console.log("L'âge est valide");
}

try {
  verifierAge(15);
} catch (error) {
  if (error instanceof AgeInvalideError) {
    console.error(`${error.name} : ${error.message} (${error.date})`);
  } else {
    console.error(`Une erreur s'est produite : ${error.message}`);
  }
}
```

Nous avons créé une classe d'erreur personnalisée `AgeInvalideError` qui étend la classe Error. Nous avons ajouté une propriété name et une propriété date pour ajouter des informations supplémentaires à l'erreur. Dans la fonction `verifierAge`, si l'âge est inférieur à 18, nous lançons une erreur personnalisée en utilisant notre classe `AgeInvalideError`. Dans le bloc `catch`, nous vérifions si l'erreur est une instance de `AgeInvalideError`, et si c'est le cas, nous affichons des informations supplémentaires sur l'erreur.

En utilisant `throw`, nous pouvons personnaliser la gestion des erreurs dans notre application et fournir des informations supplémentaires pour aider à diagnostiquer les problèmes.

finally

La structure `finally` permet d'exécuter du code quels que soient les résultats des blocs `try` et `catch`. Elle est souvent utilisée pour libérer des ressources.

Exemple

Exemple de code avec `finally` :

```
try {
  let x = 10;
  let y = x.toUpperCase();
  console.log(y);
} catch (error) {
  console.log("Une erreur s'est produite : " + error.message);
} finally {
  console.log("Fin de l'exécution du bloc try...catch.");
}
```

On voit ici que le bloc `finally` contient une instruction qui s'exécutera quoi qu'il arrive, que ce soit après le bloc `try` ou le bloc `catch`.

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice : Quiz

[solution n°2 p.14]

Question 1

La structure `try...catch` permet d'exécuter du code, quels que soient les résultats des blocs `try` et `catch`.

- ○ Vrai
- ○ Faux

Question 2

Quel est le rôle de la structure de contrôle d'erreur `try...catch` en JavaScript ?

- ○ Elle permet d'exécuter une instruction tout en gérant les erreurs potentielles
- ○ Elle permet de générer une erreur personnalisée
- ○ Elle permet d'exécuter du code quels que soient les résultats des blocs `try` et `catch`

Question 3

Quelle est la structure de contrôle d'erreur en JavaScript qui permet de générer une erreur personnalisée ?

- ○ `try...catch`
- ○ `throw`
- ○ `finally`

Question 4

Les erreurs personnalisées en JavaScript sont générées avec une seule structure.

- ○ Vrai
- ○ Faux

Question 5

La structure `catch` est obligatoire dans la structure `try...catch` en JavaScript.

- ○ Vrai
- ○ Faux

IV. Essentiel

Dans ce cours sur la gestion des erreurs en JavaScript, nous avons vu comment traiter les erreurs de syntaxe et d'exécution. Les erreurs de syntaxe sont causées par une erreur dans le code, tandis que les erreurs d'exécution se produisent lorsque le code ne peut pas être exécuté correctement. Pour traiter ces erreurs, nous avons appris à utiliser la structure `try...catch`, qui permet d'exécuter une instruction tout en gérant les erreurs potentielles.

Nous avons également vu comment utiliser la structure `throw` pour générer une erreur personnalisée. Cela nous permet de signaler des erreurs spécifiques à l'application et de fournir des informations supplémentaires pour aider à diagnostiquer les problèmes.

Nous avons également abordé les erreurs tierces. Ces erreurs se produisent dans des bibliothèques tierces que nous utilisons dans notre code. Nous avons vu comment gérer ces erreurs en utilisant les blocs `catch` et `finally`.

Pour finir, nous avons vu comment créer des classes d'erreur personnalisées en étendant la classe Error. Cela nous permet de définir des erreurs personnalisées avec des propriétés et des méthodes spécifiques à notre application.

Tous droits réservés à STUDI - Reproduction interdite

Pour conclure, la gestion des erreurs en JavaScript est une compétence essentielle pour tout développeur. En utilisant les structures de contrôle d'erreur telles que `try...catch` et `throw`, ainsi que les classes d'erreur personnalisées, nous pouvons rendre nos applications plus robustes et fiables pour les utilisateurs.

V. Auto-évaluation

A. Exercice

Vous devez réaliser une calculatrice en JavaScript. Vous avez déjà le HTML, le CSS, et une partie de JS déjà prêts, mais vous devez le terminer.

Le HTML :

```html
<!DOCTYPE html>
<html>
<head>
  <title>Calculatrice en JavaScript</title>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
  <h1>Calculatrice en JavaScript</h1>
  <div class="calculatrice">
    <label for="num1">Premier nombre:</label>
    <input type="text" id="num1" name="num1">

    <label for="operator">Opérateur:</label>
    <input type="text" id="operator" name="operator"/>

    <label for="num2">Deuxième nombre:</label>
    <input type="text" id="num2" name="num2">

    <button id="validButton" type="submit">Calculer</button>
  </div>

  <div id="result"></div>

  <script src="script.js"></script>
</body>
</html>
```

Le CSS :

```css
body {
  font-family: Arial, sans-serif;
  background-color: #f5f5f5;
}

h1 {
  text-align: center;
  margin-top: 30px;
}

.calculatrice {
  max-width: 500px;
  margin: 0 auto;
  background-color: #fff;
  padding: 20px;
  border-radius: 5px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
```

Tous droits réservés à STUDI - Reproduction interdite

```css

label {
  display: block;
  margin-bottom: 10px;
}

input[type="text"], select {
  padding: 10px;
  border-radius: 5px;
  border: 1px solid #ccc;
  font-size: 16px;
  width: 100%;
  box-sizing: border-box;
}

button[type="submit"] {
  background-color: #008CBA;
  color: #fff;
  padding: 10px;
  border: none;
  border-radius: 5px;
  font-size: 16px;
  cursor: pointer;
  transition: all 0.3s ease-in-out;
}

button[type="submit"]:hover {
  background-color: #006B87;
}

#result {
  text-align: center;
  margin-top: 20px;
  font-size: 24px;
}
```

Le JavaScript :

```js
const btn = document.querySelector('#validButton');
const resultDiv = document.querySelector('#result');

btn.addEventListener('click', (event) => {
  //Catcher les erreurs possibles pour afficher l'erreur dans resultDiv
    const num1 = validateNumber(document.querySelector('#num1').value);
    const num2 = validateNumber(document.querySelector('#num2').value);
    const operator = validateOperator(document.querySelector('#operator').value, num2);
    const result = calculate(num1, num2, operator);
    resultDiv.textContent = result;

});

function validateNumber(num) {
  //Vérifier que num est bien un nombre, sinon lancer une erreur
}

function validateOperator(operator, num2) {
  //Si l'opérateur est /, vérifier que num2 n'est pas 0, sinon lancer une erreur

  //Vérifier que l'opérateur est bien + - * / sinon lancer une erreur
```


Tous droits réservés à STUDI - Reproduction interdite

```
}

function calculate(num1, num2, operator) {
  switch (operator) {
    case '+':
      return num1 + num2;
    case '-':
      return num1 - num2;
    case '*':
      return num1 * num2;
    case '/':
      return num1 / num2;
  }
}
```

Question 1

[solution n°3 p.15]

Implémentez la fonction `validateNumber` qui prend en entrée un nombre et vérifie s'il est valide. Si le nombre est invalide, une erreur est levée avec un message d'erreur personnalisé. Sinon, le nombre est retourné.

Question 2

[solution n°4 p.15]

Implémentez la fonction `validateOperator` qui prend en entrée un opérateur et le deuxième nombre et vérifie s'il est valide. Si l'opérateur est invalide ou la division par zéro se produit, une erreur est levée avec un message d'erreur personnalisé. Sinon, l'opérateur est retourné.

Ensuite, *catchez* les erreurs pour les afficher à l'utilisateur.

B. Test

Exercice 1 : Quiz

[solution n°5 p.16]

Question 1

Quelle est la syntaxe d'une instruction try-catch en JavaScript ?

- ○ `try { // code à exécuter } finally { // code à exécuter }`
- ○ `try { // code à exécuter } catch (error) { // code à exécuter }`
- ○ `try { // code à exécuter } else { // code à exécuter }`

Question 2

La structure `try...catch` permet d'exécuter une instruction tout en gérant les erreurs potentielles.

- ○ Vrai
- ○ Faux

Question 3

La structure `finally` en JavaScript permet d'exécuter du code quels que soient les résultats des blocs `try` et `catch`.

- ○ Vrai
- ○ Faux

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Les erreurs personnalisées en JavaScript peuvent être créées en étendant la classe Error.

- Vrai
- Faux

Question 5

La gestion des erreurs est peu utilisée dans le monde professionnel.

- Vrai
- Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 5 **Solution n°1**

Question 1

Qu'est-ce qu'une erreur en JavaScript ?

- ○ Une fonctionnalité de JavaScript qui permet de contrôler le flux d'exécution du code
- ○ Un message d'avertissement qui s'affiche lorsqu'une opération est effectuée avec des données non valides
- ◉ Un arrêt de l'exécution du code en raison d'un comportement imprévu ou d'une condition invalide

Une erreur en JavaScript est un arrêt de l'exécution du code en raison d'un comportement imprévu ou d'une condition invalide. Les erreurs peuvent être causées par des erreurs de syntaxe, des erreurs de logique, des erreurs de type ou des erreurs de données.

Question 2

Les erreurs de syntaxe en JavaScript se produisent lorsque nous avons fait une erreur dans l'écriture de notre code (faute de frappe, mot-clé qui n'existe pas dans le langage, etc.).

- ◉ Vrai
- ○ Faux

Vrai. Les erreurs de syntaxe se produisent lorsque le code ne respecte pas les règles de la syntaxe du langage. Cela peut inclure des erreurs telles que des parenthèses manquantes, des guillemets manquants ou des virgules mal placées.

Question 3

Qu'est-ce qu'une erreur tierce en JavaScript ?

- ○ Une erreur qui se produit dans notre propre code
- ◉ Une erreur qui se produit dans une bibliothèque tierce que nous utilisons dans notre code
- ○ Une erreur qui se produit dans le navigateur du client

Une erreur qui se produit dans une bibliothèque tierce que nous utilisons dans notre code. Les erreurs tierces sont des erreurs qui se produisent dans des bibliothèques tierces que nous utilisons dans notre code. Ces erreurs peuvent être difficiles à diagnostiquer, car elles sont souvent liées à la bibliothèque et non à notre code.

Question 4

On peut résoudre une erreur tierce dans notre propre code.

- ○ Vrai
- ◉ Faux

Faux. Les erreurs tierces se produisent dans des bibliothèques tierces que nous utilisons dans notre code, donc on ne pourra pas la résoudre nous-mêmes.

Question 5

Quelle est la différence entre une erreur de syntaxe et une erreur d'exécution en JavaScript ?

Tous droits réservés à STUDI - Reproduction interdite

- [x] Une erreur de syntaxe se produit lorsqu'une instruction est mal écrite, tandis qu'une erreur d'exécution se produit lorsqu'une instruction est exécutée avec des données non valides
- [ ] Une erreur de syntaxe se produit lorsqu'une instruction est exécutée avec des données non valides, tandis qu'une erreur d'exécution se produit lorsqu'une instruction est mal écrite
- [ ] Il n'y a pas de différence entre les 2 types d'erreurs

Une erreur de syntaxe se produit lorsqu'une instruction est mal écrite, tandis qu'une erreur d'exécution se produit lorsqu'une instruction est exécutée avec des données non valides. Les erreurs de syntaxe sont détectées avant l'exécution du code, tandis que les erreurs d'exécution sont détectées pendant l'exécution du code.

Exercice p. 8 **Solution n°2**

Question 1

La structure `try...catch` permet d'exécuter du code, quels que soient les résultats des blocs `try` et `catch`.

- [ ] Vrai
- [x] Faux

Faux. La structure `finally` permet d'exécuter du code quels que soient les résultats des blocs `try` et `catch`. Le bloc `finally` est souvent utilisé pour libérer des ressources.

Question 2

Quel est le rôle de la structure de contrôle d'erreur `try...catch` en JavaScript ?

- [x] Elle permet d'exécuter une instruction tout en gérant les erreurs potentielles
- [ ] Elle permet de générer une erreur personnalisée
- [ ] Elle permet d'exécuter du code quels que soient les résultats des blocs `try` et `catch`

Elle permet d'exécuter une instruction tout en gérant les erreurs potentielles. La structure `try...catch` permet d'essayer d'exécuter une instruction tout en capturant les erreurs potentielles. Si une erreur se produit, elle est capturée et gérée dans le bloc `catch`.

Question 3

Quelle est la structure de contrôle d'erreur en JavaScript qui permet de générer une erreur personnalisée ?

- [ ] `try...catch`
- [x] `throw`
- [ ] `finally`

La structure `throw` permet de générer une erreur personnalisée. Elle est souvent utilisée pour signaler des erreurs spécifiques à l'application.

Question 4

Les erreurs personnalisées en JavaScript sont générées avec une seule structure.

- [ ] Vrai
- [x] Faux

Tous droits réservés à STUDI - Reproduction interdite

Les erreurs personnalisées en JavaScript peuvent être créées en étendant la classe Error pour définir des erreurs personnalisées avec des propriétés et des méthodes spécifiques à notre application, mais également avec la structure `throw`.

Question 5

La structure `catch` est obligatoire dans la structure `try...catch` en JavaScript.

- (●) Vrai
- ( ) Faux

Vous avez toujours besoin d'un bloc `catch`, mais il peut être vide et vous n'avez pas besoin de passer de variable. Si vous ne voulez pas du tout de bloc `catch`, vous pouvez utiliser `try / finally`, mais notez qu'il n'avalera pas les erreurs comme le fait un `catch` vide, donc finalement peu d'intérêt à utiliser ce modèle.

p. 11 **Solution n°3**

```
function validateNumber(num) {
    num = parseFloat(num);
    if (isNaN(num)) {
      throw new Error('Les deux paramètres d\'entrée doivent être des nombres.');
    }
    return num;
  }
```

p. 11 **Solution n°4**

```
const btn = document.querySelector('#validButton');
  const resultDiv = document.querySelector('#result');

  btn.addEventListener('click', (event) => {
    try {
        const num1 = validateNumber(document.querySelector('#num1').value);
        const num2 = validateNumber(document.querySelector('#num2').value);
        const operator = validateOperator(document.querySelector('#operator').value, num2);
        const result = calculate(num1, num2, operator);
        resultDiv.textContent = result;
    } catch (error) {
      resultDiv.textContent = error.message;
    }
  });

  function validateNumber(num) {
    num = parseFloat(num);
    if (isNaN(num)) {
      throw new Error('Les deux paramètres d\'entrée doivent être des nombres.');
    }
    return num;
  }

  function validateOperator(operator, num2) {
    if (operator === '/' && num2 === 0) {
      throw new Error('Impossible de diviser par zéro.');
    }

    switch (operator) {
```

Tous droits réservés à STUDI - Reproduction interdite

```
      case '+':
      case '-':
      case '*':
      case '/':
        return operator;
      default:
        throw new Error('Opération invalide.');
    }
  }

  function calculate(num1, num2, operator) {
    switch (operator) {
      case '+':
        return num1 + num2;
      case '-':
        return num1 - num2;
      case '*':
        return num1 * num2;
      case '/':
        return num1 / num2;
    }
  }
```

Exercice p. 11 **Solution n°5**

Question 1

Quelle est la syntaxe d'une instruction try-catch en JavaScript ?

- ○ `try { // code à exécuter } finally { // code à exécuter }`
- ◉ `try { // code à exécuter } catch (error) { // code à exécuter }`
- ○ `try { // code à exécuter } else { // code à exécuter }`

L'instruction `try` est utilisée pour exécuter un bloc de code susceptible de générer une erreur, tandis que l'instruction `catch` est utilisée pour gérer les erreurs en spécifiant un bloc de code à exécuter si une erreur se produit.

Question 2

La structure `try...catch` permet d'exécuter une instruction tout en gérant les erreurs potentielles.

- ◉ Vrai
- ○ Faux

Vrai. La structure `try...catch` permet d'essayer d'exécuter une instruction tout en capturant les erreurs potentielles. Si une erreur se produit, elle est capturée et gérée dans le bloc `catch`.

Question 3

La structure `finally` en JavaScript permet d'exécuter du code quels que soient les résultats des blocs `try` et `catch`.

- ◉ Vrai
- ○ Faux

Tous droits réservés à STUDI - Reproduction interdite

Vrai. La structure `finally` permet d'exécuter du code quels que soient les résultats des blocs `try` et `catch`. Elle est souvent utilisée pour libérer des ressources.

Question 4

Les erreurs personnalisées en JavaScript peuvent être créées en étendant la classe Error.

- (●) Vrai
- ( ) Faux

Vrai. Les erreurs personnalisées en JavaScript peuvent également être créées en étendant la classe Error pour définir des erreurs personnalisées avec des propriétés et des méthodes spécifiques à notre application.

Question 5

La gestion des erreurs est peu utilisée dans le monde professionnel.

- ( ) Vrai
- (●) Faux

Faux. La gestion des erreurs est fondamentale en programmation. Elle évitera des comportements inattendus, et évitera aux utilisateurs de voir des messages techniques. Elle évite aussi que l'application crashe, ce qui est très désagréable pour l'utilisateur.

Tous droits réservés à STUDI - Reproduction interdite